

TASK 1:

TASK 2:

1. What is Linux?

Linux is an open-source operating system based on the Linux kernel, created by Linus Torvalds in 1991. It provides a robust, secure, and versatile platform for computers, servers, smartphones, and other devices. Unlike proprietary OSes like Windows or macOS, Linux is freely available, and its source code can be modified and redistributed by anyone.

2. Variations of Linux

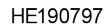
Linux distributions (distros) are different versions of Linux tailored for specific needs. Some popular distros include:

- Ubuntu — user-friendly desktop and server OS.
- Fedora — cutting-edge technology, often used by developers.
- Debian — known for stability and vast software repositories.
- CentOS / AlmaLinux / Rocky Linux — enterprise-focused, usually based on Red Hat.
- Arch Linux — flexible and customizable, favored by advanced users.

Linux Mint — easy to use, especially for newcomers.

3. Ubuntu

Ubuntu is one of the most popular Linux distros, developed by Canonical Ltd. It aims to be user-friendly with an intuitive interface, making it ideal for beginners and desktops. Ubuntu is based on Debian, providing a large software repository, regular updates, and strong community support. It's widely used in both personal computers and servers, and offers different flavors, including Ubuntu Desktop, Server, and specialized versions like Ubuntu Core for IoT devices.



hieu@Hieu: /mnt/C/Users/Put x + v

GNU nano 7.2 HE190797.txt *

Class:SE2029

Name : Nguyễn Thạch Hiếu

File Name to Write: HE190797.txt

^G Help ^C Cancel ^O DOS Format ^M Mac Format ^A Append ^B Prepend ^X Backup File ^T Browse

TASK 3:

1) Discuss about Mainframe Operating Systems. Give some examples for the OS.

=> Mainframe operating systems are designed for room-sized computers commonly used in major corporate data centers. These systems differ from personal computers primarily in their high I/O capacity, with configurations such as 1000 disks and millions of gigabytes of data. Mainframes are increasingly used as high-end Web servers and for large-scale electronic commerce and business-to-business transactions.

These operating systems focus on processing many jobs simultaneously, most requiring large amounts of I/O. They provide three main services:

1. **Batch processing** – handles routine jobs without user interaction, like claims processing in insurance companies.
2. **Transaction processing** – deals with many small requests per second, such as bank check processing or airline reservations.
3. **Timesharing** – supports multiple remote users running jobs concurrently, like querying large databases.

+> An example of a mainframe operating system is **OS/390, OS/360**.

However, these systems are gradually being replaced by UNIX variants like **Linux**.

6) Discuss about Real-Time Operating Systems. Give some examples for this type of OS?

Real-Time Operating Systems (RTOS) are systems where time is a key parameter. In industrial process-control systems, real-time computers collect data and control machines.

+> Hard real-time systems require strict deadlines—for example, a welding robot on a car assembly line must act at a precise moment; failure to do so can ruin the product. These systems are used in industrial control, avionics, and military applications and must guarantee actions within strict timing.

+> Soft real-time systems tolerate occasional missed deadlines without permanent damage. Examples include digital audio, multimedia systems, and smartphones.

In some hard real-time systems, the OS may be a tightly coupled library with no protection between parts. An example of this is **eCos**.

Real-time systems overlap with embedded and handheld systems. Most have soft real-time features, run only designer-installed software, and are primarily used in industrial contexts.

Examples of RTOS:

- eCos
- embedded OS in industrial, avionics, and military systems.

TASK 4: List 11 steps in making the system call read(fd, buffer, nbytes) in the lecture's reference book?

1. Push nbytes

2. Push & bufer

3.Push fd

4. Call read

5. Put code for read in register

6.Trap to the kernel

7.Dispatch

8.System call handler

9. Return to library procedure

10.Return to caller

11. Increment SP

TASK 5: Summarize the section 1.7 (pages 63 to 73) in the reading book ?

1.7 Operating System Structure

This section discusses different architectural approaches to building operating systems.

Monolithic Systems (1.7.1)

- The entire operating system runs as a single program in kernel mode.
- It is organized as a collection of procedures that can call each other freely.
- Services are grouped into a single binary, making performance good but maintenance difficult.

- Modularity can be introduced with clearly defined interfaces between system components.

Layered Systems (1.7.2)

- The operating system is divided into layers, each built on top of the lower one.
- Each layer only interacts with the layer directly beneath it.
- This structure simplifies debugging and system verification but can reduce efficiency due to layer overhead.

Microkernels (1.7.3)

- Only essential core services (like IPC, basic scheduling, and memory management) run in the kernel.
- Most services (file systems, device drivers, etc.) run in user space as servers.
- This model enhances modularity and reliability but can suffer from performance issues due to message-passing overhead.

Client-Server Model (1.7.4)

- Extends the microkernel approach by treating the OS as a set of servers offering services to client processes.
- The communication is done via well-defined protocols, typically message passing.
- It supports distributed systems and is scalable and robust.

Virtual Machines (1.7.5)

- Provide an abstraction that allows multiple operating systems to run concurrently on the same hardware.
- Each virtual machine appears as a real machine to the software.
- Used in environments where resource isolation and operating system experimentation are important.

Exokernels (1.7.6)

- Aim to give as much control as possible to application-level software.
- The kernel is minimal, managing only resource protection and multiplexing.
- Applications or libraries implement traditional OS abstractions tailored to specific needs, offering high flexibility and performance.